

Sorry ransomware exploits cPanel authentication bypass

 threatlocker.com/blog/sorry-ransomware-exploits-cpanel-authentication-bypass

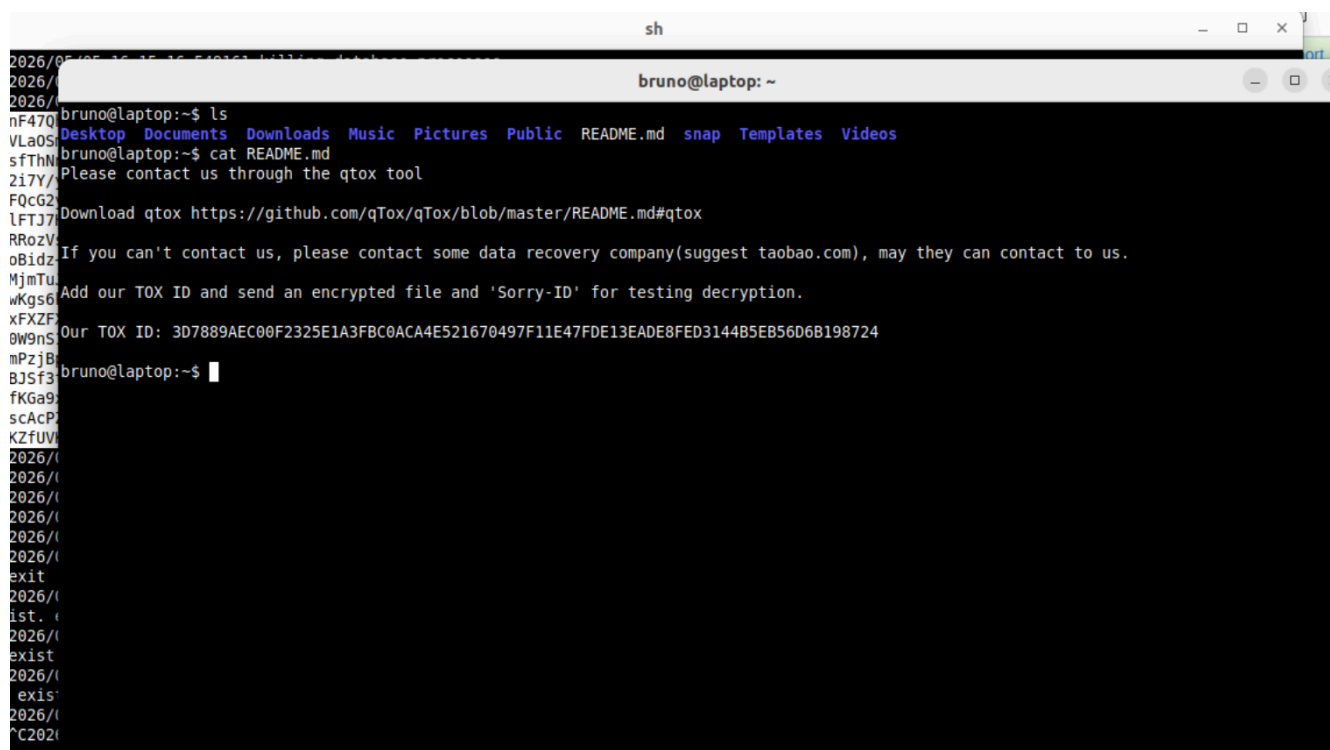
May 15, 2026

John Moutos, William Pires, Pandeli Zi, Alex Aguiar

An emerging ransomware group by the name of “Sorry” has recently surfaced to capitalize on the cPanel authentication bypass vulnerability (CVE-2026-41940). The exploit involves compromising web servers and deploying a Linux encryptor written in Golang, which appends the extension “.sorry” to affected files.

This ransomware group is taking an opportunistic approach by exploiting the recently disclosed CVE within 48 hours of its disclosure, targeting Linux servers running cPanel/WHM or WP Squared with a vulnerability in the cPanel login process.

The group has not disclosed a Tor leak site but rather has been directing victims to a Tox-based chat for negotiations via the ransom note, README.md. Alternatively, they also suggest contacting data recovery companies on Taobao, a subsidiary of Alibaba, for assistance.



```
sh
bruno@laptop: ~
bruno@laptop:~$ ls
Desktop Documents Downloads Music Pictures Public README.md snap Templates Videos
bruno@laptop:~$ cat README.md
Please contact us through the qtox tool
Download qtox https://github.com/qTox/qTox/blob/master/README.md#qtox
If you can't contact us, please contact some data recovery company(suggest taobao.com), may they can contact to us.
Add our TOX ID and send an encrypted file and 'Sorry-ID' for testing decryption.
Our TOX ID: 3D7889AEC00F2325E1A3FBC0ACA4E521670497F11E47FDE13EAD8FED314485EB56D6B198724
bruno@laptop:~$
```

Figure 1: Ransom Note

CVE-2026-41940 is vulnerable to CRLF injection attack

The primary avenue that Sorry ransomware has been deployed is through vulnerable cPanel versions between version 11.42, released on March 4, 2014, and the multiple patched versions released on May 1, 2026. These versions are vulnerable to a CRLF injection attack that allows unauthenticated users to write server-side session files without proper sanitization.

CRLF (Carriage Return Line Feed), which refers to the injected characters that allow unexpected actions.

Carriage return (\r) and line feed (\n) are used universally to signify the end of a line or command, and they are both expected to be parsed correctly in any kind of user input. Unfortunately, when an authorization request is sent to a cPanel web server with these characters encoded in base64, they are not stripped from the input. Several other intended features are used to parse, write, and use the payload decorated with these characters.

Three-part exploit

The exploit is performed in three parts: a capture of the session cookie, the actual CRLF injection, and forcing the server to use the injected values.

The session cookie that is captured does not come from a specially crafted request or the vulnerability itself. If a login is attempted with nonexistent credentials, a session file is created on the server to track this login attempt. Creating a pre-auth session file is standard for web servers and is used to track ongoing and failed login attempts for very valid reasons. The session name is returned as a cookie in response to this failed login attempt.

The CRLF injection attack is now used to write to that pre-auth session file. A GET request is crafted so that the authentication header uses “root” as a user, and the password as the intended payload, which includes the incorrectly parsed “\r” and “\n” characters and specific values to be placed in the session file.

As a result, values such as “hasroot=1\r\n”, “tfa_verified=1\r\n”, and “user=root\r\n” are saved to the attacker’s session file.

A few steps remain to complete the attack as these malicious values (including \r and \n) are still saved within the “pass” field in the session file and not as actual entries yet.

To parse the CRLF characters and write these values as entries, a separate control flow is triggered by a GET request with a missing security token. This flow decides not to use the cached version of the session file and parses the injected values into their respective JSON

entries. The pre-auth session file has now been populated with injected values; however, the login authentication process is still checking the provided password against `/etc/shadow`, which contains the hashed password to root.

To bypass this, a final value is added to the injected payload, which is now being parsed as intended. If a `“successful_internal_auth_with_timestamp”` entry is recognized, internal authentication is assumed by the server, and the password is not checked against the shadow file.

With this assumed internal authentication and values such as `“hasroot”` and `“tfa_verified”` injected into the pre-auth file, an attacker can successfully bypass authentication and act as root. This attack began with no foothold other than unauthenticated access to the cPanel server.

Although this attack is being actively exploited, its discovery was by no means simple, and it combines a single unsanitized input with several arguably standard practices to completely bypass authentication and act as the root user.

Figure 2: Sorry Attack Chain

This privileged access is then used to deploy the Sorry encryptor.

Successful encryption on compromised servers

The Sorry ransomware binary is written in Golang and targets hosts running Linux operating systems. It is distribution agnostic to ensure encryption success on compromised servers hosting web content.

Figure 3: Detect It Easy

Execution mutex

On initial execution, the encryptor creates a `“Sorry.lock”` file in the `/tmp` directory which appears to be a mutex, blocking additional instances of the encryptor from running simultaneously. The Linux syscall `“flock”` is called with the `LOCK_EX` and `LOCK_NB` flags, meaning that an exclusive nonblocking file lock is attempted. Interestingly, if this fails, the error is logged to the console, and the binary continues with encryption anyways. Despite this clearly being execution mutex functionality, it does not work as apparently intended and allows for two instances of the encryptor to run at once, potentially causing several workflow problems.

Figure 4: Lock File Creation

Pre-encryption

Once the mutex is created, the encryptor creates a unique victim ID composed of the current username, hostname, CPU count, OS string, active network interfaces, and the path the encryptor is executing out of. This data is joined with “|” characters, added to a fake user agent, and communicated through a TCP connection to attacker infrastructure hosted at IP 68[.]183[.]190[.]253.

Figure 6: C2 Infrastructure

Several services and processes are then terminated by using `systemctl stop <svc>`, `service <svc> stop`, and `kill -9 <svc>`.

Targeted services include:

- mysqld
- oracle
- postgresql
- mariadb
- mongod
- redis
- redis-server
- cassandra
- elasticsearch
- sshd

Target processes include:

- agntsvcagntsvc
- agntsvcencsvc
- agntsvcisqlplussvc
- dbeng50
- dbsnmp
- excel
- firefoxconfig
- infopath
- mongod
- mongos
- msaccess
- msftesql

- mspub
- mydesktopqos
- mydesktopservice
- mysql
- mysqld
- mysqld-nt
- mysqld-opt
- ocautoupds
- ocomm
- ocssd
- onenote
- ora_pmon
- ora_smon
- oracle
- outlook
- postgres
- postmaster
- powerpnt
- redis-server
- sqbcoreservice
- sqlagent
- sqlbrowser
- sqlservr
- sqlwriter
- steam
- synctime
- tbirdconfig

The encryptor config is constructed and populated with default embedded values, including the TOX ID for negotiation, appended file extension, ransom note filename, excluded paths, excluded filenames, and target extensions.

Figure 7: Config Defaults

Excluded filenames and paths include:

- README.md
- thumbs.db
- netuser.dat
- autorun.inf

- iconcache.db
- bootfont.bin
- desktop.ini
- .ds_store
- node_modules
- .git
- cache
- __pycache__
- site-packages
- /bin/
- /boot/
- /dev/
- /sys/
- /etc/
- /usr/
- /run/
- /lib/
- /proc/
- /lost+found/

Figure 8: Excluded Filenames

After the encryptor config is fully constructed, the user's home directory is checked for the marker file “.sorry_exist”, and if not found, it is created. This is an empty secondary marker file showing that an instance of the encryptor has been executed, and it is used to prevent additional encryption passes on remote hosts accessed through SSH. If this file exists, the encryptor halts and exits.

Figure 9: Marker File

Three layers of encryption

The local encryption process starts by creating a new session-specific RSA key/pair combination. The target file contents are encrypted using AES-GCM, and the 32-byte AES key is encrypted with the public RSA key created previously and written to the encrypted file's header.

To decrypt each file, the session RSA private key is used to decrypt the file-specific AES key within the header, which is then used to decrypt the file. Although this may seem trivial, a final step is taken to encrypt that RSA private key. An embedded public key is used to encrypt the private key created locally, and that encrypted private key is eventually written to the artifact

“~/sorry_id_<timestamp>.sorry”. The private key belonging to this embedded value is held by the attackers and is required to recover the locally generated session RSA private key, which can then decrypt AES keys and allow for decryption of individually encrypted files.

A visualization of this process helps to understand the three layers of encryption taken.

Figure 10: Encryption Process

The encryptor walks target directories starting from the host root “/” by default unless target directories are specified as an argument on launch. Collected directories are then deduplicated and assigned to workers to target files based on a selection of prioritized file extensions and types. Files such as documents and databases are targeted first before moving on to a much broader selection of file types, including but not limited to applications, media, virtual hard disks, scripts, source code, development secrets, and development artifacts. Target file paths for each included directory are queued for each worker, and the per-file encryption process is started.

When detonating this sample, several logs are written to the console along with a lengthy base64 string. This string is the session-specific RSA private key mentioned earlier and is joined with two other facts: the hostname and an MD5 hash of the host profile string. This combined value is encrypted by the embedded public key, encoded in base64, and saved to the “sorry_id_<timestamp>.sorry” file, which is referred to in the ransom note as needed by the attackers to decrypt example files.

```
<SESSION RSA PRIVATE KEY>|hostname|MD5 of (build_host_profile_string()) RSA  
encryption from embedded RSA public key
```

Encoded in base64

Written to sorry_id_<timestamp>.sorry

Figure 11: sorry_id_<timestamp>.sorry

Figure 12: Constructing Encrypted Recovery Blob

SSH propagation

Extensive effort is made to spread the encryptor through all available local SSH connections, as well as any mapped external IP addresses. Targets are accumulated by levels 0-3:

0. Exact host targets from ~/ .ssh/known_hosts* and /etc/hosts
1. Hosts expanded from active local subnets
2. /24 expansion from discovered IPv4 prefixes

3. /16 expansion from discovered IPv4 prefixes

Level 0 is the only level that would add external addresses to the target list, but it entirely depends on addresses mapped in host files. This level allows the encryptor to possibly leave the local network and compromise a completely different set of machines.

Level 1 takes the subnets assigned to active network interfaces and expands them to all possible non-loopback addresses. From these local subnets, /24 and /16 expansions are taken in levels 2 and 3. Although these groups may completely overlap with active /24 and /16 subnets expanded from level 1, an active subnet with a different range, for example /8, would contain thousands of addresses that /24 and /16 would not contain.

The reason that level 2 exists is not clear, since a /24 subnet would always be included in the /16 subnet covered by level 3. Once the target list is built and deduplicated, the attack begins.

Figure 13: SSH Target Scanning

This feature then scans the constructed target list for SSH services on ports 22, 2222, and 22222. For all detected SSH services, a dispatcher function begins a dictionary attack on multiple machines at once with a hardcoded list of usernames and passwords. The following username and password entries are used by default to access networked hosts.

Username:

- root
- admin
- test
- ubuntu
- oracle
- postgres
- guest
- user
- kali

Passwords:

- <empty>
- root
- toor
- 123456
- password

- <user>
- <user>123
- test
- ubuntu
- oracle
- postgres
- guest
- user
- kali

Once an attack is successful and an SSH connection is established, the encryptor copies and executes itself onto the target machine.

No master list of compromised hosts is shared, and the new execution is performed as if it were the first. This means that every host that runs the encryptor would attempt to attack SSH on all available hosts and would need to succeed and locate the “.sorry_exist” marker file before noticing that the host had already been encrypted.

This decision to attack already compromised machines wastes a significant amount of time and resources and severely negates the benefits of network-wide encryption, as the generated login failures would be easily discoverable by attentive security teams.

Although this dictionary attack is rather straightforward, SSH protections are often placed on externally facing SSH ports, and attacks made internally may go unnoticed for as long as needed. Without specific safeguards in place, stopping malicious connections from several unconfirmed machines would be a serious challenge.

Finally, the system “flock” placed on the execution mutex file “sorry.lock” is released, and both the encryptor and the lock file are deleted.

Timely patching is critical as attackers move quickly

The Sorry ransomware campaign demonstrates how quickly an opportunistic threat actor can weaponize vulnerabilities and gain root level access to exposed servers.

Sorry's exploitation of CVE-2026-41940 combined with widespread SSH attacks amplifies the overall impact across environments. The core risk remains for organizations that are unaware or slow to patch systems who can be compromised and encrypted with minimal effort from attackers.

Organizations should prioritize timely patching, access controls, and monitoring to reduce exposure and limit lateral movement and damage.

Figure 14: Publicly Hosted Victims

IOCs

IPs

68[.]183[.]190[.]253

Files

~/./sorry_exist

README.md

~/sorry_id_<19 digit timestamp>.sorry

/tmp/Sorry.lock

2fc0a056fd4eff5d31d06c103af3298d711f33dbcd5d122cae30b571ac511e5a

TOX ID

3D7889AEC00F2325E1A3FBC0ACA4E521670497F11E47FDE13EADE8FED3144B5EB56D6B198724

ABOUT THE AUTHOR

John Moutos, threat intelligence team lead at ThreatLocker, is a cybersecurity specialist with roots in enterprise penetration testing and offensive security research. John studied at the Oregon Institute of Technology and earned his bachelor's from the SANS Technology Institute. At ThreatLocker, he leads the threat intelligence program, turning research into practical detections and guidance that helps organizations anticipate and counter emerging attacks.

ABOUT THE AUTHOR

William Pires, threat analyst at ThreatLocker, brings a data-first mindset shaped by hands-on work in MDR and customer-facing threat response. He holds a bachelor's degree in accounting and spent two years as a credit analyst before moving into cybersecurity, a path that sharpened his rigor, accuracy, and judgment under pressure. He advanced from approval

requests to MDR, then to threat analysis, where he pieces together attack paths from confirmed incidents and researches active threats, group profiles, and emerging tactics. At ThreatLocker, William focuses on translating research into clear detections and practical guidance for customers. His recent work spans software communication analysis, incident triage, and providing cutting edge threat intelligence to enterprise environments and the cybersecurity industry.

ABOUT THE AUTHOR

Pandeli Zi, threat analyst at ThreatLocker, brings a lifelong curiosity for how systems work, an instinct that started with taking things apart just to see why they functioned the way they did. That curiosity led him into technology, where early exposure to cybersecurity concepts, data security, and digital risk shaped into a bachelor's degree from UCF. Before joining ThreatLocker, Pandeli built technical depth through a range of IT and asset management roles at organizations including Orlando Health, Florida's Turnpike Enterprise, and Morgan & Morgan. At ThreatLocker, he focuses on daily threat tracking, malware research, and post-incident analysis, translating intelligence into practical insights that strengthen customer defenses.

ABOUT THE AUTHOR

Alexander Aguiar, Threat Analyst at ThreatLocker, is passionate about advancing security awareness through research, technical analysis and real-world application of defense strategies. He holds a bachelor's degree in Criminal Justice as well as a Master's degree in Cybersecurity & Privacy from the University of Central Florida. He began his career as a security behavioral analyst at Universal Studios Orlando, where he developed a strong foundation in threat awareness and anomaly detection. In response to the growing cyber threat landscape, he pivoted to the security analyst role at ThreatLocker. His work emphasizes translating complex threat activity into clear, actionable insights that strengthen organizational defense.